

PROYECTO INTEGRADOR

EcoMunicipio GT

Institución: Centro Educativo Técnico Laboral Kinal

Curso / Área: Taller

Código Técnico: IN5CM

Docente: Eduardo Humberto Hor López

Equipo / Integrantes:

Angel Gabriel Portillo Raymundo - 2022361

Luis Enrique Chac Gonzales -2025089

Esteban David Chun Escobar - 2025155

Domingo Alejandro Romero Hernández - 2025157

Fecha de inicio: 12/08/26

Fecha de finalización:

ETAPA 1

Nombre del proyecto

EcoMuni GT

sistema ciudadano para el reporte y seguimiento de problemas ambientales relacionados con residuos.

Problemática

Guatemala presenta dificultades en el manejo de residuos sólidos, entre ellas la acumulación de basura en espacios públicos, basureros clandestinos, quema de desechos y contaminación de áreas comunes. Según la ENIGH 2022-2023, el 45.6 % de los hogares reportó quemar su basura y solo el 3.1 % utilizó aboneras o reciclaje como forma de disposición. Esta situación evidencia la necesidad de fortalecer la información, el reporte y el seguimiento ciudadano de problemas ambientales.

Actualmente, los reportes pueden realizarse por distintos canales, como llamadas, mensajes o redes sociales. Esto dificulta centralizar evidencias, identificar ubicaciones, evitar duplicados y conocer el estado posterior de cada caso.

Justificación

EcoMuni GT permitirá concentrar en una sola plataforma los reportes ciudadanos relacionados con residuos y contaminación de espacios públicos. El sistema no sustituirá a las municipalidades ni a los mecanismos oficiales de denuncia; funcionará como una herramienta de registro, seguimiento, información ambiental y participación ciudadana.

Usuarios afectados

- Habitantes de comunidades urbanas y rurales.
- Vecinos cercanos a basureros clandestinos o zonas contaminadas.
- Municipalidades y encargados de gestión ambiental.
- Organizaciones ambientales, voluntarios y centros de reciclaje.

Situación actual

La información sobre problemas ambientales suele encontrarse dispersa y sin un seguimiento uniforme. Entre las principales dificultades están:

- Falta de un canal centralizado para registrar reportes.
- Dificultad para adjuntar evidencia y ubicación precisa.
- Escaso seguimiento visible para el ciudadano.
- Desconocimiento de puntos de reciclaje y campañas ambientales.

Solución

Desarrollar una aplicación web que permita registrar problemas ambientales mediante descripción, categoría, ubicación y fotografías. Los reportes podrán ser revisados por usuarios autorizados y avanzar por estados de seguimiento: Reportado, En revisión, Verificado, En atención y Resuelto. También podrá utilizarse el estado Rechazado cuando el reporte sea falso, duplicado o insuficiente.

La plataforma incluirá además consulta de reportes, puntos de reciclaje, campañas ambientales, notificaciones, estadísticas y un panel administrativo.

Objetivo general

Desarrollar un sistema web que permita registrar, organizar, consultar y dar seguimiento a reportes ciudadanos sobre residuos y contaminación de espacios públicos, facilitando la participación ciudadana y el acceso a información ambiental

ETAPA 2 — Planificación del proyecto

Planificación

Semana 1	Definir alcance el alcance que obtendrá el proyecto , usuarios, roles, módulos, requerimientos funcionales y no funcionales. Analizar el problema y organizar las tareas del equipo. Crear repositorio y tablero de trabajo.
Semana 2	Diseñar arquitectura del sistema. Crear modelo entidad-relación y estructura en MySQL, para luego ser trasladada a PostgreSQL. Definir tablas para cada entidad correspondiente con sus atributos ya definidos.
Semana 3	Configurar proyecto con Node.js y TypeScript. Crear conexión con Base de Datos. Implementar modelos y primeras rutas de API REST. Implementar usuarios y roles.
Semana 4	Desarrollar CRUD de reportes. Implementar categorías, descripción, fotografías, ubicación y estados. Crear las principales vistas de Angular: Login, registro, creación de reporte y consulta de reportes.
Semana 5	Conectar completamente Angular con API REST. Implementar un historial de reportes, comentarios y apoyo. Desarrollar puntos de reciclaje, campañas y ubicación. Crear un panel administrativo/home.
Semana 6	Realizar pruebas funcionales, validaciones, permisos y seguridad. Probar diferentes roles y escenarios. Detectar reportes duplicados, errores de formularios y problemas de navegación. Corregir errores y optimizar rendimiento.
Semana 7	Preparar versión final. Configurar variables, respaldos y configuración del sistema. Realizar pruebas finales. Elaborar documentación técnica, guía de instalación y manual de usuario. Preparar demostración y presentación.

Actividades

1. Análisis: Investigar la problemática, definir usuarios, alcance y requerimientos.
2. Base de datos: Diseñar entidades, atributos, relaciones y modelo entidad-relación.
3. Diseño: Crear prototipos, navegación, formularios y paneles de usuario.
4. Backend : Configurar Node.js, Express y TypeScript; desarrollar API REST, validaciones, autenticación y conexión a base de datos.
5. Frontend: Desarrollar interfaces en Angular, formularios, servicios y consumo de API.
6. Integración : Conectar frontend, backend y base de datos; verificar operaciones y permisos.
7. Pruebas: Ejecutar pruebas funcionales y de integración, registrar errores y corregirlos.
8. Implementación: Preparar una versión funcional, configurar variables de entorno y verificar el sistema.
9. Documentación: Elaborar documentación técnica, manual de usuario y guía de instalación.

10. Mantenimiento: Corregir fallos posteriores y proponer mejoras futuras.

Recursos, herramientas y tecnologías

- Frontend: Angular y TypeScript.
- Backend: Node.js, Express y TypeScript.
- Base de datos: Postgre.
- API: arquitectura REST.
- Control de versiones: Git y GitHub.
- Gestión de tareas: Trello.
- Herramientas: Visual Studio Code, Terminal, pnpm, Angular CLI, Postman y navegador web.
- Recurso opcional: Leaflet y OpenStreetMap para visualizar ubicaciones.

Riesgos

1. Reportes falsos o inapropiados: Aplicar revisión y moderación antes de considerarlos verificados.
2. Reportes duplicados: Detectar coincidencias por ubicación y categoría.
3. Acceso no autorizado: Utilizar autenticación, contraseñas protegidas y permisos por roles.
4. Pérdida de información: Realizar respaldos y manejo adecuado de errores.
5. Dependencia de Internet: Optimizar interfaces e imágenes para reducir consumo de datos.
6. Alcance excesivo: Limitar la primera versión a los módulos definidos como prioritarios.
7. Falta de atención institucional: Aclarar que el sistema organiza y da seguimiento, pero no garantiza la resolución física del problema.

ETAPA 3 — Análisis de requerimientos

¿Qué necesita el Usuario?

Mediante la observación y la investigación permitieron identificar que en Guatemala existen problemas frecuentes relacionados con la acumulación de basura, los basureros clandestinos y la disposición inadecuada de residuos.

A partir de los resultados, se concluyó que los usuarios necesitan una herramienta que facilite reportar problemas ambientales de manera rápida y organizada, permitiendo agregar fotografías, descripción, categoría y ubicación.

Asimismo, se identificó la necesidad de proporcionar información sobre puntos de reciclaje y campañas ambientales, además de contar con mecanismos para revisar y validar los reportes, reduciendo casos falsos o duplicados.

En conclusión, EcoMuni GT responde a una necesidad real, ya que busca centralizar los reportes ambientales y facilitar su seguimiento, promoviendo una mayor participación ciudadana y mejor organización de la información. La plataforma funcionará como una herramienta de apoyo y seguimiento, sin sustituir las funciones oficiales de las municipalidades.

Necesidades del usuario

- Registrar un problema de forma rápida y sencilla.
- Adjuntar ubicación y fotografías como evidencia.
- Consultar el estado y el historial de sus reportes.
- Acceder a información clara sobre reciclaje y campañas ambientales.
- Utilizar la plataforma desde computadora o teléfono con seguridad y privacidad.

Requerimientos Funcionales

- Registrar usuarios, iniciar sesión y asignar roles.
- Crear reportes con categoría, ubicación, descripción y fotografías.
- Consultar y filtrar reportes por categoría, estado o ubicación.
- Actualizar el estado y conservar el seguimiento de cada reporte.
- Permitir que los usuarios apoyen y comenten reportes.
- Administrar puntos de reciclaje y campañas ambientales.
- Mostrar estadísticas y supervisar el sistema.

Requerimientos no funcionales

- Proteger contraseñas, datos personales y accesos según el rol.
- Validar la información antes de guardarla.
- Mantener un funcionamiento rápido y estable.
- Adaptar la interfaz a computadoras, tabletas y teléfonos.
- Mantener una navegación sencilla y un código organizado.

ETAPA 4 — Historias de usuario

Historia no.1: Como ciudadano me gustaría poder registrar un problema relacionado con residuos o contaminación, esto sería útil para informar de manera rápida sobre situaciones que afectan nuestro entorno/comunidad/residencial, entre otros.

Historia no.2: Como ciudadano, el poder adjuntar fotografías, describir la situación y la ubicación del problema. Esto sería útil para proporcionar evidencia que permita identificar con mayor facilidad la situación reportada. También permitiría brindar información más completa sobre el lugar afectado. De esta manera, el reporte tendría mayor claridad para su revisión.

Historia no.3: Como ciudadano, el poder consultar el estado de los reportes que he realizado sería útil. Esto sería útil para conocer si un problema está reportado, en revisión, verificado o en atención. También podría saber cuándo un reporte ha sido resuelto o rechazado. Así tendría mayor conocimiento sobre el seguimiento de cada caso.

Historia no.4: Como ciudadano, me gustaría poder consultar el historial de los reportes que he realizado. Esto sería útil para conocer las actualizaciones y cambios que ha tenido cada reporte. También permitiría revisar la información registrada anteriormente. De esta manera, podría mantener un mejor control sobre los problemas informados.

Historia no.5: Como ciudadano, me gustaría poder buscar y filtrar reportes por categoría, estado o ubicación. Esto sería útil para encontrar de manera rápida información sobre problemas ambientales específicos. También facilitaría la consulta de reportes registrados en diferentes lugares. Así podría conocer situaciones que afectan determinadas zonas o comunidades.

Historia no.6: Como ciudadano, me gustaría poder comentar y apoyar reportes realizados por otros usuarios. Esto sería útil para aportar información adicional sobre problemas que también afectan a mi comunidad. Además, permitiría mostrar apoyo a situaciones que requieren atención. De esta forma, se fomentaría una mayor participación ciudadana.

Historia no.7: Como administrador, me gustaría poder revisar, validar y actualizar los reportes realizados por los ciudadanos. Esto sería útil para mantener información confiable dentro de la plataforma. También permitiría identificar reportes falsos, duplicados o que no cuenten con información suficiente. Así se podría dar un seguimiento adecuado a cada caso.

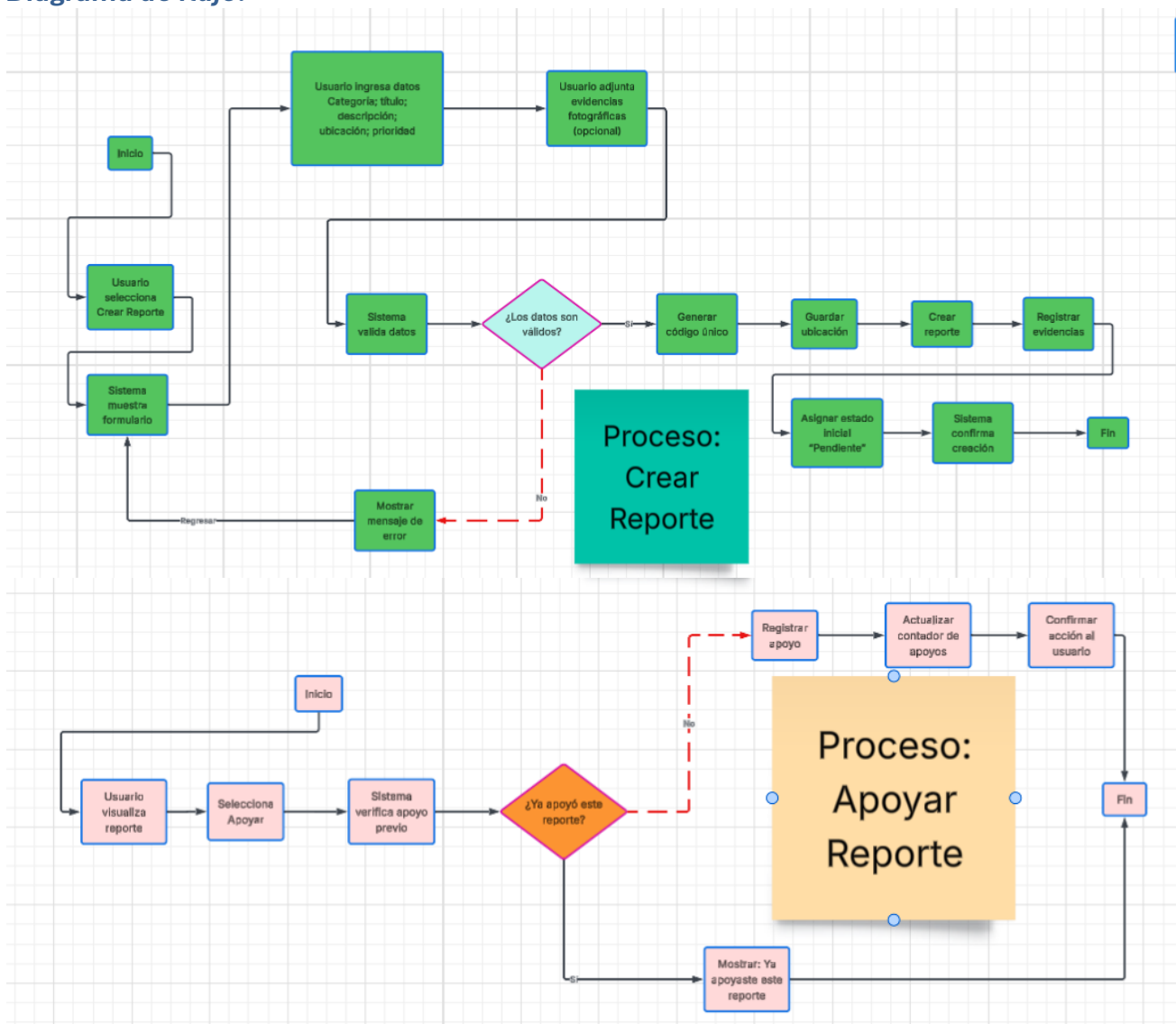
Historia no.8: Como administrador, me gustaría poder administrar los puntos de reciclaje y las campañas ambientales. Esto sería útil para mantener actualizada la información disponible para los ciudadanos. También permitiría agregar, modificar o retirar información cuando sea necesario. De esta manera, la plataforma ofrecería información ambiental más confiable.

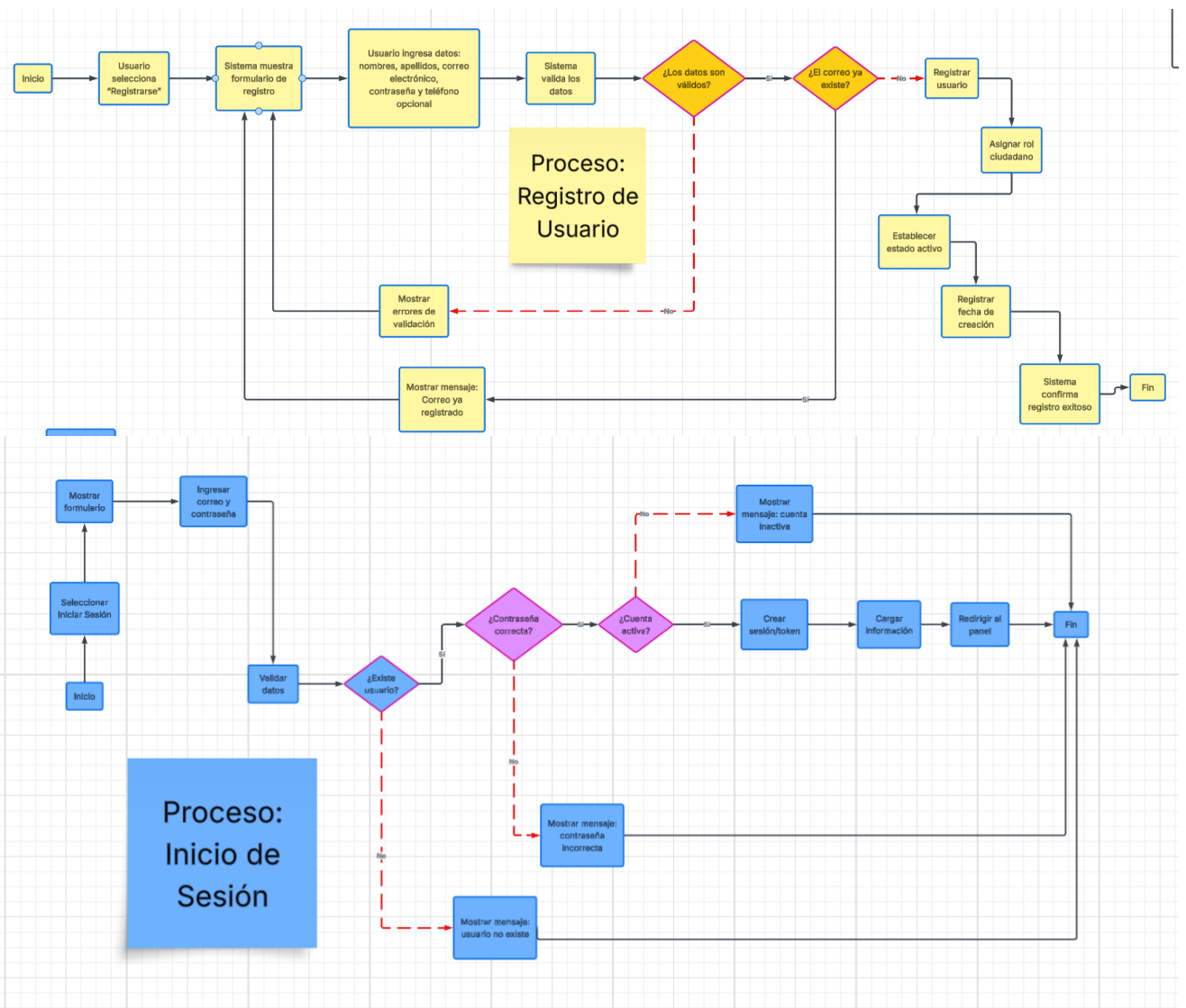
Historia no.9: Como ciudadano, me gustaría poder consultar los puntos de reciclaje y las campañas ambientales disponibles. Esto sería útil para conocer alternativas que permitan manejar correctamente los residuos. También facilitaría encontrar actividades ambientales en las que pueda participar. Así se promovería una mayor conciencia y participación dentro de la comunidad.

Historia no.10: Como administrador, me gustaría poder consultar estadísticas de los reportes registrados en la plataforma. Esto sería útil para conocer la cantidad y situación de los problemas ambientales reportados. También permitiría identificar categorías, estados o zonas con mayor cantidad de reportes. De esta manera, se facilitaría la supervisión y análisis de la información.

ETAPA 5 — Diseño del sistema

Diagrama de flujo:





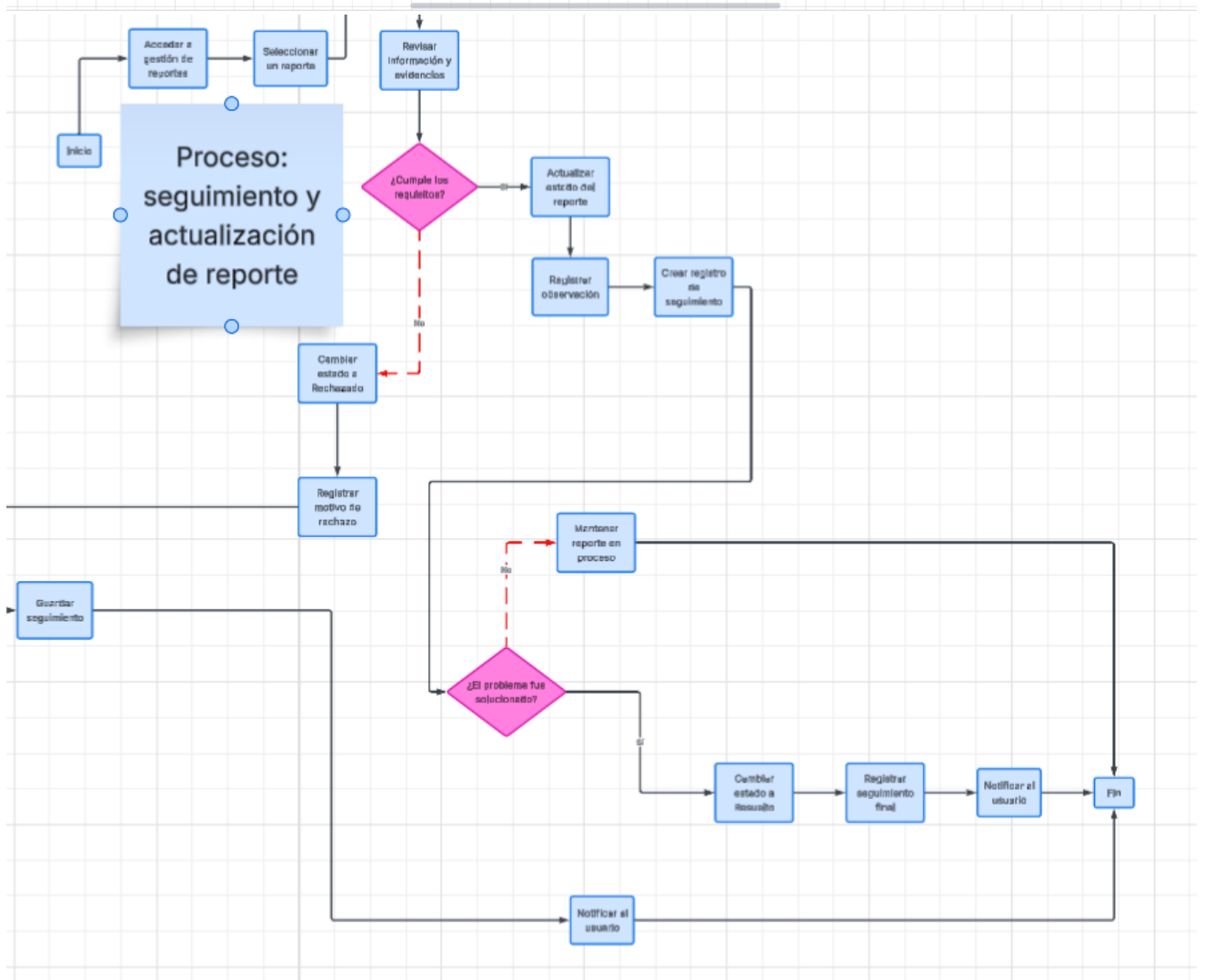
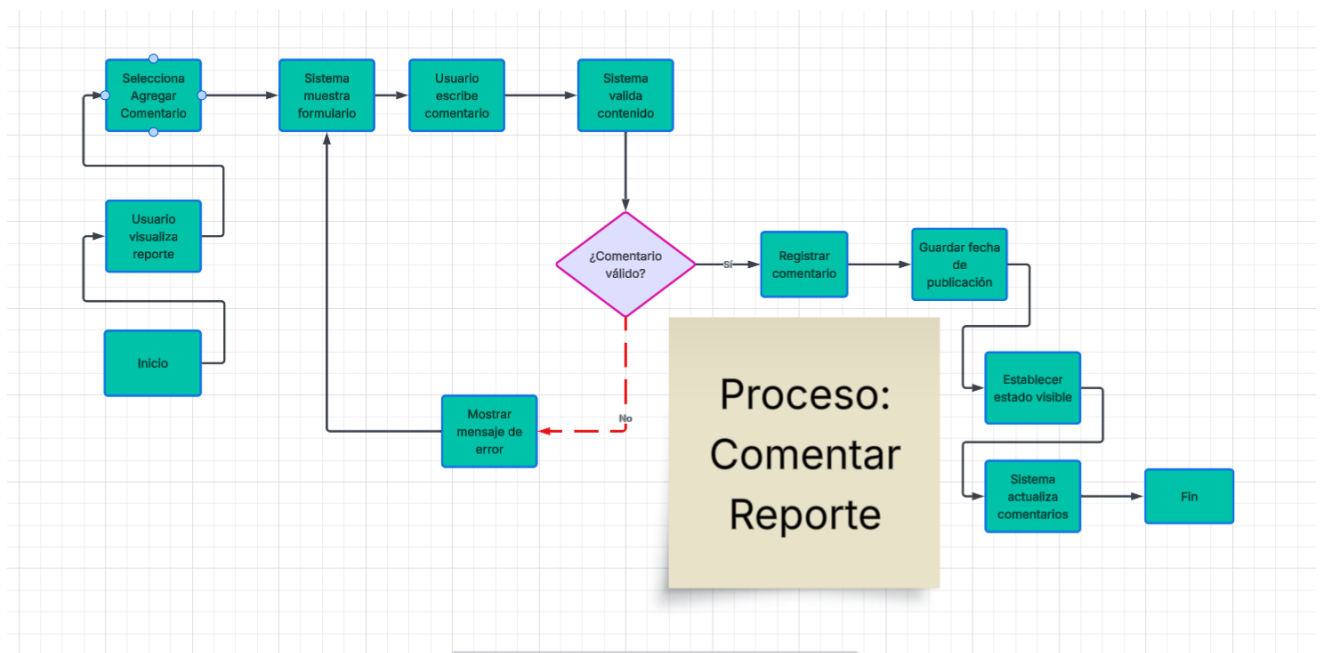
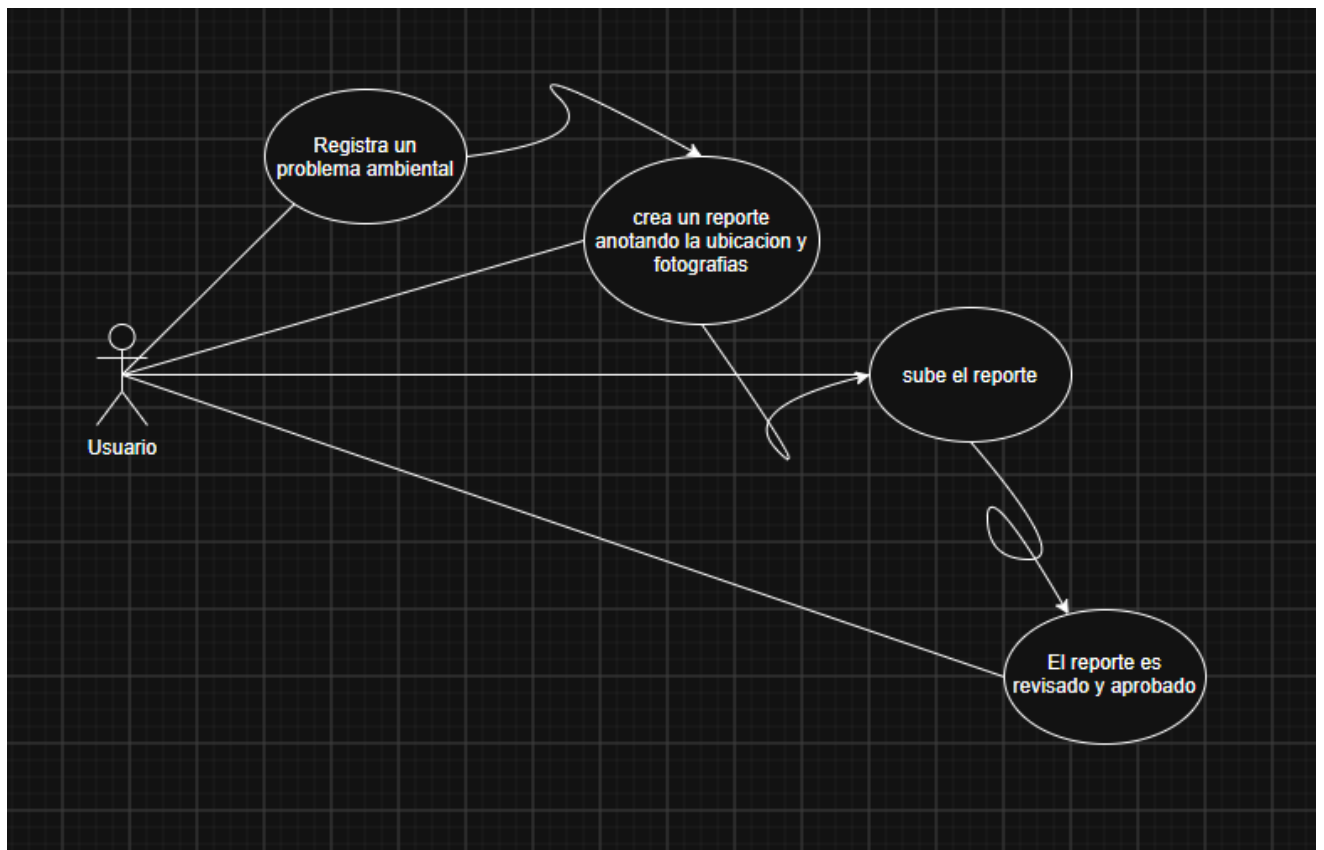


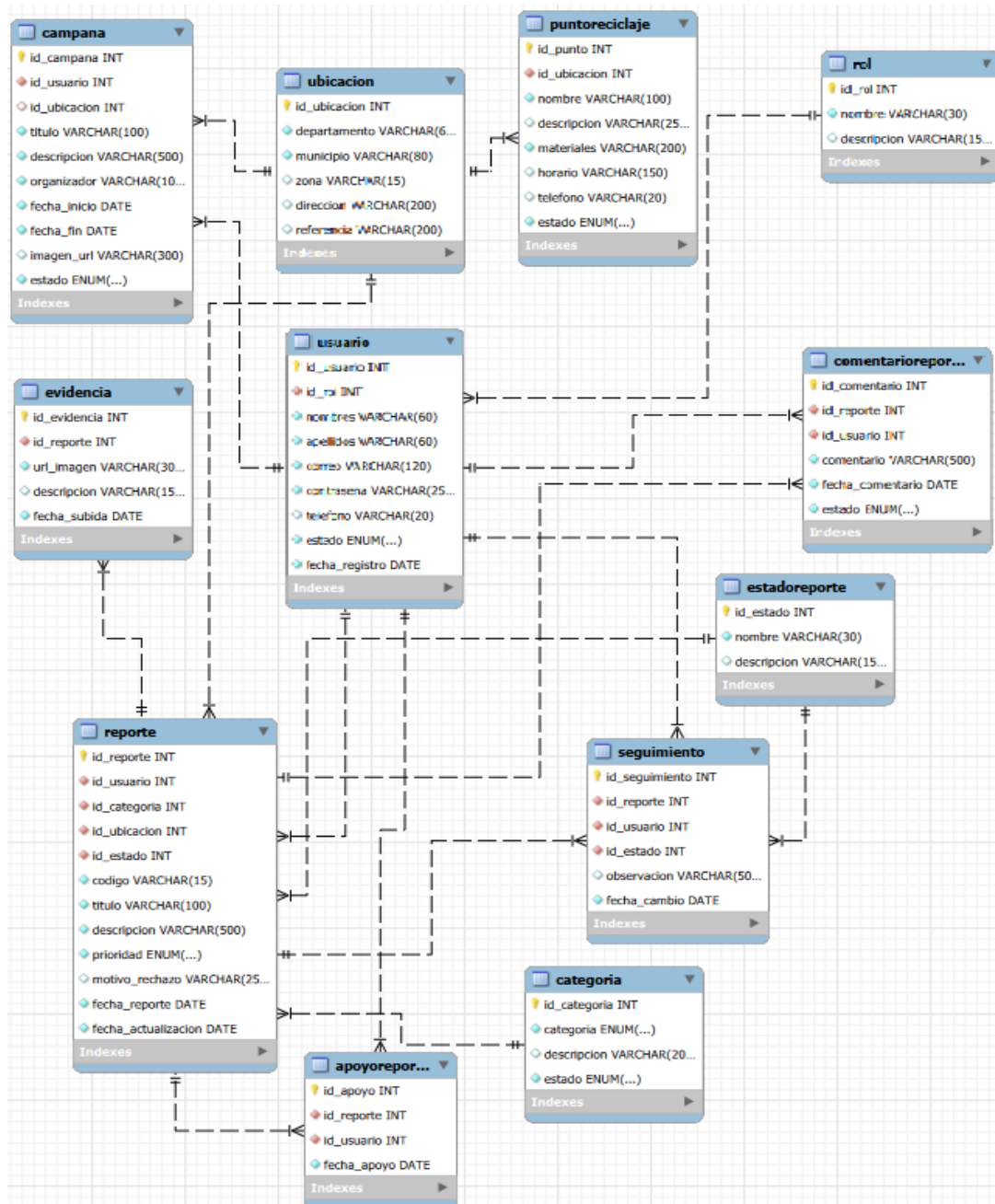
Diagrama de casos de uso:



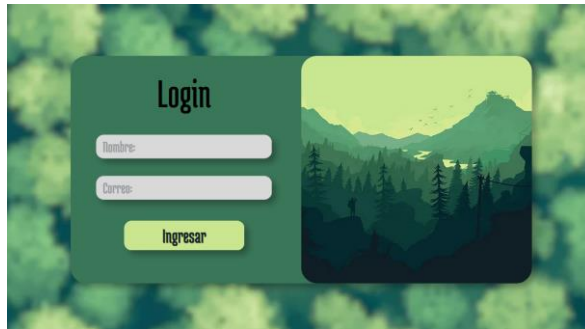
Arquitectura del sistema: Cliente (Angular / Web) --> (Peticones HTTP) --> Servidor (Node.js + Express) --> (Consultas SQL) --> Base de Datos (PostgreSQL)

ETAPA 6 — Diseño de base de datos

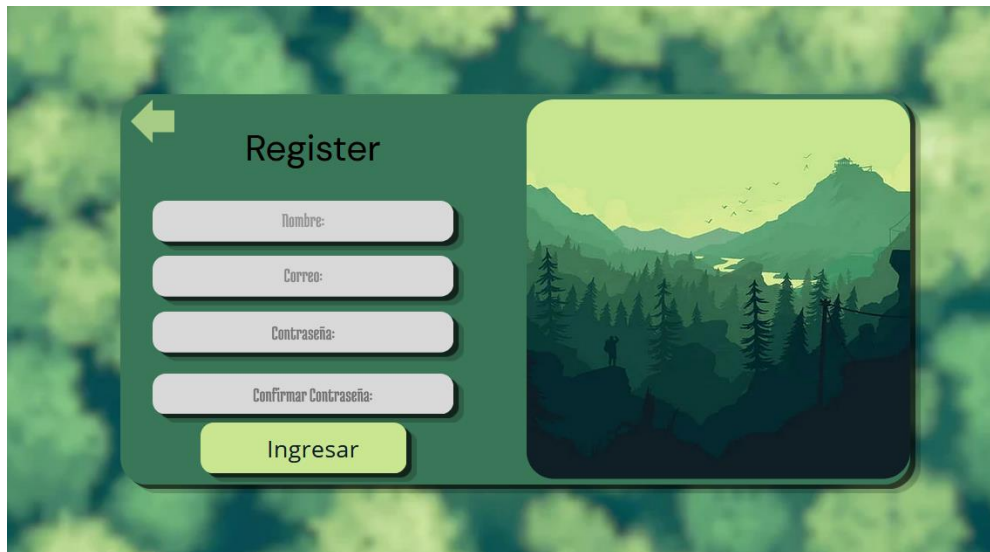
Diagrama Entidad Relación:



ETAPA 7 — Diseño de interfaces



- Login



- Register



- Ubicaciones



- Home

ETAPA 8 — Desarrollo del software

1. Desarrollo general del sistema

Durante esta etapa se realizó la programación funcional de **EcoMuni GT**, integrando el frontend desarrollado en Angular, un backend basado en Node.js y Express, y una base de datos en PostgreSQL.

La aplicación permite que los usuarios puedan registrarse, iniciar sesión, navegar entre las diferentes secciones del sistema y consultar información relacionada con reportes ambientales, campañas y puntos de reciclaje.

Además, el sistema implementa operaciones de creación, consulta y actualización de información, validaciones, manejo de errores y control de permisos según el rol del usuario.

La arquitectura implementada sigue el siguiente flujo:

Usuario → Angular → Servicios HTTP → Backend → Servicios → PostgreSQL

Esta separación permite mantener organizada la aplicación y distribuir las responsabilidades entre la interfaz, la lógica del sistema y la persistencia de datos.

2. Autenticación y control de acceso

EcoMuni implementa un sistema de autenticación basado en sesiones.

El usuario puede:

- Registrarse en el sistema.
- Iniciar sesión mediante correo y contraseña.
- Mantener una sesión activa.
- Cerrar sesión.
- Acceder únicamente a rutas autorizadas.

Angular utiliza un AuthService para comunicarse con el backend y conservar la información del usuario autenticado. Las solicitudes de autenticación se realizan mediante HTTP y utilizan credenciales para permitir el funcionamiento de la sesión.

Las rutas principales también utilizan un authGuard, encargado de comprobar si existe una sesión activa antes de permitir el acceso a páginas protegidas como Home, Campañas y los departamentos.

En el backend, el inicio de sesión valida las credenciales, comprueba la contraseña mediante bcrypt y genera una sesión asociada al usuario.

3. Navegación del sistema

La navegación del frontend se desarrolló mediante Angular Router.

Actualmente existen rutas para:

- Inicio de sesión.
- Registro.
- Home.
- Campañas.
- Alta Verapaz
- Baja Verapaz
- Chimaltenango
- Chiquimula
- El Progreso
- Escuintla
- Guatemala
- Huehuetenango
- Izabal
- Jalapa
- Jutiapa
- Petén
- Quetzaltenango

- Quiché
- Retalhuleu
- Sacatepéquez
- San Marcos
- Santa Rosa
- Sololá
- Suchitepéquez
- Totonicapán
- Zacapa

Las rutas que requieren autenticación están protegidas mediante `authGuard`. Los componentes de los departamentos utilizan además carga diferida mediante `loadComponent`.

Desde la página principal, el usuario puede seleccionar los departamentos disponibles y acceder a su contenido específico.

4. CRUD y gestión de reportes

Uno de los principales módulos desarrollados corresponde a la gestión de reportes ambientales.

Desde los departamentos disponibles, los usuarios pueden:

- Consultar reportes.
- Crear nuevos reportes.
- Visualizar el detalle de un reporte.
- Agregar comentarios.
- Apoyar o retirar el apoyo de un reporte.
- Adjuntar evidencias.
- Consultar el seguimiento.
- Filtrar reportes.

El servicio del frontend centraliza las peticiones HTTP necesarias para comunicar Angular con el backend.

Por su parte, el backend implementa las operaciones mediante rutas específicas para reportes, comentarios, apoyos, seguimientos y evidencias.

La lógica relacionada con los reportes se encuentra separada en servicios que realizan consultas SQL, verificaciones y transacciones en PostgreSQL.

5. Validación de datos

El sistema incluye validaciones tanto en frontend como en backend.

Entre las principales validaciones implementadas se encuentran:

- Verificación de campos obligatorios.

- Validación de correo electrónico.
- Validación de contraseñas.
- Validación de categorías.
- Validación de prioridades de reportes.
- Verificación de estados.
- Validación de fechas.
- Restricción de tamaño y formato de imágenes.
- Control de permisos según el usuario.

En el módulo de evidencias, por ejemplo, solamente se aceptan archivos PNG, JPEG o WebP con un tamaño máximo de 5 MB.

El backend también verifica el contenido real del archivo y no solamente su extensión, aumentando la seguridad del sistema.

6. Manejo de errores y mensajes al usuario

EcoMuni implementa manejo de errores en distintas capas.

El backend devuelve códigos HTTP y mensajes dependiendo del problema ocurrido, por ejemplo:

- 400: datos inválidos.
- 401: usuario no autenticado.
- 403: usuario sin permisos.
- 404: recurso no encontrado.
- 409: conflicto de información.
- 413: archivo demasiado grande.
- 500: error interno.

El servidor incluye un middleware general de manejo de errores para devolver respuestas controladas al frontend.

Angular recibe estas respuestas y muestra mensajes comprensibles para el usuario, evitando mostrar directamente errores técnicos.

También se utilizan estados de carga y mensajes de confirmación después de operaciones exitosas, como crear un reporte, guardar una evidencia o actualizar información.

7. Base de datos

La aplicación utiliza PostgreSQL como sistema gestor de base de datos.

Entre las tablas principales se encuentran:

- Rol.
- Usuario.

- Categoría.
- Ubicación.
- EstadoReporte.
- Reporte.
- Evidencia.
- Seguimiento.
- ApoyoReporte.
- PuntoReciclaje.
- Campaña.
- ComentarioReporte.

Las tablas están relacionadas mediante llaves primarias y llaves foráneas, permitiendo conservar la integridad de la información.

La conexión con PostgreSQL se realiza desde el backend mediante la librería pg y variables de entorno para establecer host, puerto, nombre de la base de datos, usuario y contraseña.

Al iniciar el backend también se preparan automáticamente catálogos básicos como roles, categorías y estados de los reportes.

8. Formularios y consultas

La aplicación contiene formularios para registrar y modificar información.

Entre ellos se encuentran:

- Registro de usuarios.
- Inicio de sesión.
- Creación de reportes.
- Comentarios.
- Evidencias.
- Seguimiento de reportes.
- Puntos de reciclaje.
- Campañas.

Los datos ingresados desde los formularios se envían mediante servicios HTTP hacia el backend.

El sistema también realiza consultas dinámicas utilizando filtros por estado, categoría, prioridad y página.

De esta manera, el usuario puede localizar información de manera más ordenada sin necesidad de cargar todos los registros simultáneamente.

9. Arquitectura implementada

EcoMuni utiliza una arquitectura dividida en tres capas principales.

Frontend

Desarrollado con Angular y TypeScript.

Sus principales responsabilidades son:

- Mostrar la interfaz gráfica.
- Administrar formularios.
- Controlar la navegación.
- Mostrar mensajes.
- Consumir la API del backend.

Backend

Desarrollado con Node.js, Express y TypeScript.

Se encarga de:

- Recibir peticiones HTTP.
- Validar información.
- Autenticar usuarios.
- Verificar permisos.
- Procesar operaciones.
- Ejecutar consultas en la base de datos.
- Manejar errores.

La estructura del backend separa configuraciones, middleware, modelos, routers, servicios y servidor.

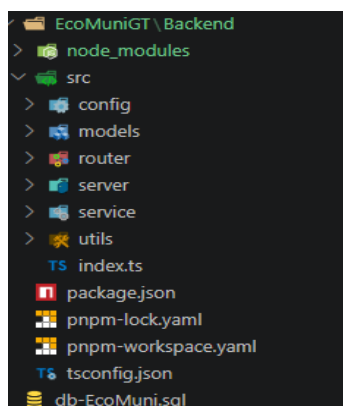
Base de datos

PostgreSQL almacena de forma persistente la información generada por el sistema.

La arquitectura completa puede representarse de la siguiente forma:

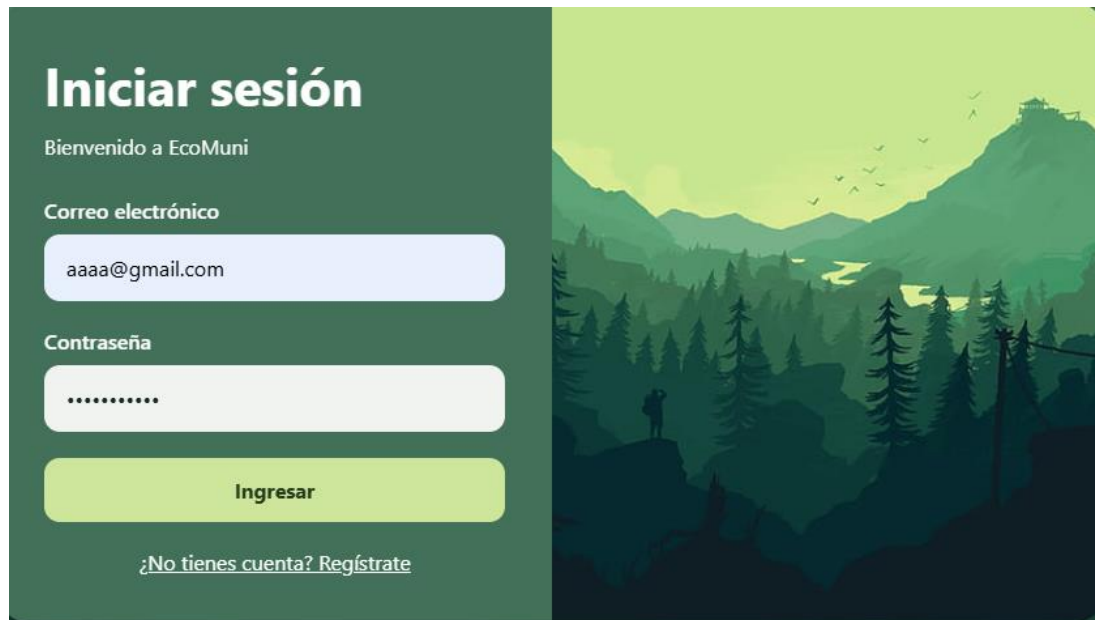
Angular → HTTP → Express → Servicios → PostgreSQL

Esta estructura permite mantener separada la interfaz gráfica de la lógica del negocio y de la información almacenada.



ETAPA 9 — Pruebas del software

- Inicio de Sesión Exitosa



- Creación de un Reporte

REPORTES AMBIENTALES

Nuevo reporte en Alta Verapaz

Título
Mucha basura acumulada en las calles

Categoría: **acumulacion de basura** | Prioridad: **Alta**

Municipio: **Cobán** | Zona — opcional: **12**

Dirección — opcional
7a avenida 9-08

Referencia — opcional
cerca del puente de madera

Descripción
se ha visto mucha basura tirada en la calle lo que ocasiona muchos malos olores y es molesto

Después de guardar podrás adjuntar hasta cinco fotografías desde el detalle del reporte.

[Cancelar](#) [Crear reporte](#)

Reporte gtu. **EcoMuni** Inicio Campañas Seguimiento **Reportes** [Cerrar sesión](#)

PARTICIPACIÓN CIUDADANA [+ Nuevo reporte](#)

Reportes ambientales

Consulta incidencias ambientales en Alta Verapaz.

Estado: **Todos** | Categoría: **Todas** | Prioridad: **Todas**

Pendiente **Alta**

acumulacion de basura

Mucha basura acumulada en las calles

+ Cobán, 12, 7a avenida 9-08

se ha visto mucha basura tirada en la calle lo que ocasiona muchos malos olores y es molesto

22/09/2026 0 0

[Ver detalle](#)

[Anterior](#) Página 1 de 1 - 1 reportes [Siguiente](#)

Pendiente

Mucha basura acumulada en las calles

R7121bd10eb4ba3

+ Cobán, 12, 7a avenida 9-08 22/09/2026 0 0

[Apoyar reporte](#)

Descripción

se ha visto mucha basura tirada en la calle lo que ocasiona muchos malos olores y es molesto

Evidencias

Este reporte todavía no tiene evidencias.

Adjuntar fotografía (PNG, JPEG o WebP; hasta 5 MB)

[Elegir archivo](#) No se ha seleccionado ningún archivo

Descripción de la fotografía — opcional

Describe lo que muestra la imagen

0/150 caracteres

[Guardar evidencia](#)

Seguimiento del reporte

Pendiente 22/09/2026

- Registro con datos Inválidos

Contraseña

.....

Mínimo 8 caracteres: mayúscula, minúscula, número y un símbolo como ! o @.

Confirmar contraseña

.....

Las contraseñas no coinciden.

Contraseña

.

Mínimo 8 caracteres: mayúscula, minúscula, número y un símbolo como ! o @.

Confirmar contraseña

.

Completa correctamente los campos obligatorios.

- Carga de evidencias

Evidencias



adjunto la evidencia de la acumulacion de basura

Editar descripción

adjunto la evidencia de la acumulacion

Guardar descripción

Adjuntar fotografía (PNG, JPEG o WebP; hasta 5 MB)

Elegir archivo No se ha seleccionado ningún archivo

Descripción de la fotografía — opcional

Describe lo que muestra la imagen

0/150 caracteres

Guardar evidencia

- Comentarios

0/150 caracteres

Guardar evidencia

Seguimiento del reporte

✓ **Pendiente** 22/09/2026
Reporte registrado.
Luis Portillo

Comentarios 2

L **Luis Portillo** 22/09/2026
grande portillo

L **Luis Portillo** 22/09/2026
era mucha basura?

Escribe un comentario...

Enviar

- Vista General de Departamentos

Reporte gu. **EcoMuni** Inicio Campañas Seguimiento **Reportes** Cerrar sesión

PARTICIPACIÓN CIUDADANA **Reportes ambientales** **+ Nuevo reporte**
Consulta incidencias ambientales en Alta Verapaz.

Estado: Todos Categoría: Todas Prioridad: Todas

Pendiente Alta
acumulacion de basura
Mucha basura acumulada en las calles
+ Cobán, 12, 7a avenida 9-08
se ha visto mucha basura tirada en la calle lo que ocasiona muchos malos olores y es molesto
22/09/2026 0 0 **Ver detalle**

Anterior Página 1 de 1 - 1 reportes Siguiente

Pendiente **Mucha basura acumulada en las calles**
R7121bd10e04ba3
+ Cobán, 12, 7a avenida 9-08 22/09/2026 0

Apoyar reporte

Descripción
se ha visto mucha basura tirada en la calle lo que ocasiona muchos malos olores y es molesto

Evidencias
Este reporte todavía no tiene evidencias.

Adjuntar fotografía (PNG, JPEG o WebP: hasta 5 MB)
Elegir archivo No se ha seleccionado ningún archivo

Descripción de la fotografía — opcional
Describe lo que muestra la imagen

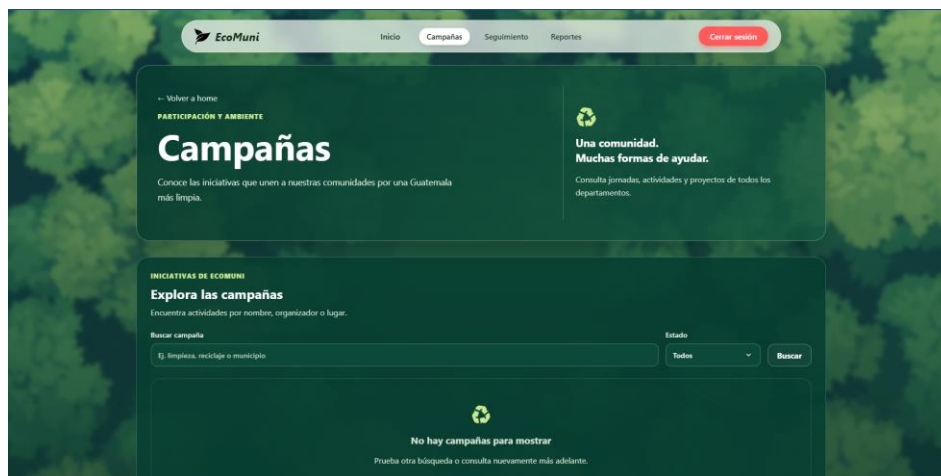
0/150 caracteres
Guardar evidencia

Seguimiento del reporte
Pendiente 22/09/2026

- Vista general de la vista departamentos/reportes



- Vista general de Campañas



ETAPA 10 — Implementación

1. Objetivo

Preparar EcoMuni GT para que otra persona pueda instalarlo y ejecutarlo correctamente siguiendo una serie de pasos.

El sistema utiliza tres partes principales:

Frontend Angular → Backend Node.js/Express → PostgreSQL

2. Software necesario

Antes de instalar el proyecto se debe contar con:

- Node.js.
- pnpm.
- PostgreSQL.
- Git.
- Visual Studio Code.
- Navegador web.

Para comprobar que Node.js, pnpm y Git están instalados:

```
node --version
pnpm --version
git --version
```

3. Descargar el proyecto

Desde una terminal ejecutar:

```
git clone https://github.com/EstebanChun-2025155/EcoMuni.git
```

Ingresar al proyecto:

```
cd EcoMuni
```

Seleccionar la rama utilizada:


```
git checkout develop
```

La estructura principal será:

```
EcoMuni/  
├── Backend/  
├── Frontend/  
└── db-EcoMuni.sql
```

4. Configurar la base de datos

EcoMuni utiliza PostgreSQL.

Crear una base llamada: `db_ecomuni`

Después ejecutar dentro de esa base el archivo: `db-EcoMuni.sql`

Este archivo crea automáticamente las tablas y relaciones necesarias para el sistema.

Entre las principales tablas se encuentran:

- Usuario.
- Rol.
- Reporte.
- Categoría.
- Ubicación.
- Evidencia.
- Comentario.
- Seguimiento.
- Campaña.
- Punto de reciclaje.

5. Configurar el Backend

Abrir una terminal dentro de: `EcoMuni/Backend`

Instalar las dependencias: `pnpm install`

Después crear un archivo: `.env` dentro de la carpeta `Backend`.

Su contenido debe ser:

PORT=3000

NODE_ENV=development

FRONTEND_ORIGIN=http://localhost:4200

DB_HOST=localhost

DB_PORT=5432

DB_NAME=db_ecomuni

DB_USER=postgres

DB_PASSWORD=CONTRASEÑA

SESSION_SECRET=Clave de 32 dígitos hexadecimales

En DB_PASSWORD debe colocarse la contraseña real de PostgreSQL.

Para generar el SESSION_SECRET ejecuta:

```
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
```

Copiar el resultado dentro del .env.

El archivo .env no debe subirse a GitHub porque contiene información privada.

6. Ejecutar el Backend

Desde la carpeta Backend:

`Pnpm run dev`

Si la conexión con PostgreSQL es correcta, el servidor quedará funcionando en:

<http://localhost:3000>

7. Configurar y ejecutar el Frontend

Abrir otra terminal dentro de: EcoMuni/Frontend

Instalar dependencias: `pnpm install`

Ejecutar Angular: `ng serve --open`

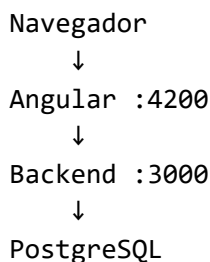
El sistema estará disponible en: <http://localhost:4200>

8. Orden para iniciar EcoMuni

Cada vez que se quiera utilizar el proyecto se debe seguir este orden:

1. Iniciar PostgreSQL.
2. Ejecutar el Backend con `pnpm dev`.
3. Ejecutar el Frontend con `ng serve --open`.
4. Abrir <http://localhost:4200>.

El flujo del sistema será:



9. Usuarios de prueba

Para comprobar los diferentes permisos del sistema pueden utilizarse usuarios de los siguientes roles:

Rol	Ejemplo de correo
Administrador	admin@ecomuni.com
Supervisor	supervisor@ecomuni.com
Ciudadano	ciudadano@ecomuni.com

Las contraseñas deben estar almacenadas cifradas en PostgreSQL.

También es posible crear nuevos usuarios ciudadanos desde la opción **Registrarse** del sistema.

10. Comprobación final

La instalación puede considerarse correcta si se logra:

1. Abrir la pantalla de inicio de sesión.
2. Iniciar sesión correctamente.
3. Entrar al Home.
4. Abrir un departamento.
5. Consultar o registrar un reporte.
6. Comprobar que la información se guarda en PostgreSQL.
7. Cerrar sesión.

11. Errores comunes

No conecta con PostgreSQL

Revisar principalmente:

```
DB_NAME=db_ecomuni  
DB_USER=postgres  
DB_PASSWORD=contraseña_correcta
```

También comprobar que PostgreSQL esté iniciado.

Error de SESSION_SECRET

La clave debe tener exactamente 64 caracteres hexadecimales.

Puede generarse nuevamente con:

```
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
```

El Frontend no carga información

Comprobar que ambos servidores estén activos:

Frontend: <http://localhost:4200>
Backend: <http://localhost:3000>

Resultado

Al completar estos pasos, EcoMuni GT queda funcionando localmente con:

Angular + Node.js/Express + PostgreSQL.

La documentación permite que una persona nueva pueda descargar, configurar e iniciar el proyecto sin necesidad de modificar el código fuente.

ETAPA 13 — Mantenimiento del software

1. Concepto e importancia del mantenimiento

El mantenimiento del software es el conjunto de actividades que se realizan después de que un sistema ha sido desarrollado, probado e implementado. Su finalidad es mantener el sistema funcionando correctamente, corregir errores, adaptarlo a nuevas necesidades y mejorar su calidad con el paso del tiempo.

El desarrollo de un sistema no termina cuando este logra ejecutarse correctamente. Durante su uso pueden aparecer problemas que no fueron detectados en las primeras pruebas, nuevas necesidades de los usuarios, cambios en las herramientas utilizadas o mejoras que permitan hacer el sistema más claro y eficiente.

En el caso de **EcoMuni GT**, el mantenimiento será importante porque el proyecto está pensado para crecer de forma progresiva. Nuevos departamentos, funcionalidades ambientales, cambios en la interfaz o modificaciones en la infraestructura pueden requerir ajustes posteriores.

Por esta razón, el mantenimiento debe considerarse como una etapa permanente dentro del ciclo de vida del software. Su correcta aplicación ayuda a conservar la estabilidad del sistema y facilita que futuras modificaciones puedan realizarse sin afectar las funciones que ya se encuentran desarrolladas.

2. Mantenimiento correctivo

El mantenimiento correctivo se realiza cuando se detecta un error en una función que ya debería trabajar correctamente. Su objetivo principal es localizar la causa del problema, aplicar una solución y comprobar que el cambio no afecte otras partes del sistema.

Los errores pueden encontrarse durante las pruebas o aparecer posteriormente durante el uso normal de la aplicación. Pueden estar relacionados con formularios, validaciones, autenticación, consultas a la base de datos, navegación, carga de archivos o mensajes mostrados al usuario.

En EcoMuni GT, un caso de mantenimiento correctivo podría presentarse si un formulario permite guardar información incompleta, si un reporte no aparece después de registrarse o si un usuario logra acceder a una función que no corresponde a su rol.

El proceso correctivo deberá considerar tres pasos básicos: identificar el problema, corregirlo y volver a ejecutar las pruebas necesarias. De esta forma, cada error encontrado permitirá mejorar progresivamente la estabilidad de la plataforma.

Este tipo de mantenimiento es especialmente importante porque permite conservar la confianza del usuario y evita que pequeños errores se conviertan en problemas mayores.

3. Mantenimiento adaptativo

El mantenimiento adaptativo consiste en modificar el sistema para que pueda continuar funcionando cuando cambian las condiciones tecnológicas o los requerimientos del proyecto.

Estos cambios no necesariamente se producen porque exista un error. En muchos casos, el sistema funciona correctamente, pero el entorno donde será utilizado cambia y obliga a realizar modificaciones.

Para EcoMuni GT, un ejemplo importante sería pasar de un entorno local a un servidor web. Actualmente el sistema trabaja mediante un Frontend, un Backend y una base de datos ejecutados de manera local. Si posteriormente se publica en Internet, será necesario adaptar configuraciones, direcciones del servidor, seguridad y manejo de sesiones.

También puede considerarse mantenimiento adaptativo la incorporación de nuevos departamentos, cambios en la estructura de los reportes, nuevas reglas de participación ciudadana o una actualización importante de Angular, Node.js o PostgreSQL.

La finalidad de este mantenimiento es evitar que el sistema quede desactualizado o deje de ser compatible con las tecnologías y necesidades que aparezcan en el futuro.

4. Mantenimiento perfectivo y preventivo

El mantenimiento perfectivo busca mejorar funciones que ya existen y que actualmente trabajan correctamente. No se centra en reparar errores, sino en hacer que el sistema sea más útil, cómodo, rápido o fácil de utilizar.

En EcoMuni GT podrían considerarse mejoras perfectivas la optimización de los filtros de reportes, una navegación más clara, formularios más sencillos, mensajes más comprensibles o una mejor presentación de campañas, evidencias y puntos de reciclaje.

Por otro lado, el mantenimiento preventivo se realiza antes de que aparezca un problema. Su finalidad es reducir riesgos futuros y mejorar la calidad interna del proyecto.

Entre las acciones preventivas pueden encontrarse la eliminación de código repetido, organización de carpetas, actualización controlada de dependencias, revisión de variables de entorno, mejora de validaciones y reutilización de funciones comunes.

EcoMuni ya utiliza una estructura compartida para varias páginas de departamentos, lo cual representa una práctica preventiva importante, ya que evita repetir la misma lógica en múltiples componentes.

Ambos tipos de mantenimiento permiten mejorar la calidad del sistema sin esperar a que ocurra una falla grave.

ETAPA 14 — Nuevas versiones

1. Concepto e importancia de las versiones

Una versión representa un estado determinado del software en un momento específico de su desarrollo. Cada vez que se realizan correcciones, mejoras o cambios importantes, el sistema puede recibir un nuevo número de versión.

El objetivo del versionado es mantener un registro ordenado de la evolución del proyecto. Esto permite identificar qué funcionalidades pertenecen a cada etapa, cuáles fueron modificadas y qué problemas fueron solucionados.

En EcoMuni GT, el uso de versiones facilitará reconocer los cambios que se realicen después de la primera versión funcional. De esta manera, el desarrollo no se presenta como un único producto terminado, sino como un sistema que puede continuar creciendo.

El control de versiones también facilita el mantenimiento, ya que permite conocer en qué momento fue incorporada una función o cuándo apareció una modificación determinada.

Además, si una actualización genera un problema, contar con versiones bien identificadas facilita localizar el cambio y comparar el funcionamiento con una versión anterior.

2. Sistema de numeración

Para EcoMuni GT se puede utilizar una estructura sencilla basada en tres niveles:

Mayor.Menor.Corrección

Ejemplo:

1.2.1

El primer número corresponde a la versión mayor y cambia cuando se realizan modificaciones importantes, como nuevos módulos, cambios grandes en la arquitectura o ampliaciones significativas del sistema.

El segundo número representa mejoras o nuevas funcionalidades que amplían el sistema sin cambiar completamente su funcionamiento principal.

El tercer número se utiliza para correcciones pequeñas, principalmente errores que no requieren agregar nuevas funciones.

Por ejemplo:

1.0.0 representa la primera versión funcional.

1.1.0 puede representar una versión con nuevas mejoras.

1.1.1 puede corresponder a una corrección menor.

2.0.0 puede utilizarse cuando el sistema incorpora cambios importantes o nuevos módulos.

Este sistema permite identificar rápidamente el nivel de cambio realizado.

3. Evolución propuesta de EcoMuni GT

La primera versión del proyecto puede identificarse como **EcoMuni GT 1.0.0**. Esta versión representa la base funcional del sistema e incluye las funciones principales desarrolladas durante el proyecto.

Entre ellas se encuentran el registro e inicio de sesión, navegación entre departamentos habilitados, gestión de reportes, comentarios, apoyos, evidencias, seguimiento, puntos de reciclaje y campañas.

Posteriormente, una versión **1.1.0** podría incluir correcciones detectadas después de las pruebas. En esta etapa podrían ajustarse validaciones, mensajes de error, formularios o procesos que necesiten mayor estabilidad.

Una versión **1.2.0** podría estar orientada a mejoras de interfaz y experiencia de usuario. Esto puede incluir una navegación más clara, mejores filtros, formularios más simples o una presentación más ordenada de la información.

Finalmente, una futura versión **2.0.0** podría representar una ampliación importante del proyecto, incluyendo más departamentos, nuevos módulos administrativos, implementación en servidores web y funciones adicionales para la gestión ambiental.

Esta evolución permite establecer una ruta clara de crecimiento para EcoMuni GT.

4. Relación entre mantenimiento y nuevas versiones

El mantenimiento y el versionado están directamente relacionados.

Cada vez que el sistema recibe una corrección, adaptación o mejora, estos cambios pueden formar parte de una nueva versión.

El proceso puede representarse de la siguiente manera:

Versión actual → mantenimiento → pruebas → nueva versión

Primero se identifica una necesidad o problema. Después se realiza el mantenimiento correspondiente. Una vez terminado el cambio, se prueba nuevamente el sistema. Si el resultado es correcto, las modificaciones pueden agruparse y publicarse como una nueva versión.

Esto permite evitar cambios desordenados y facilita saber exactamente qué contiene cada actualización.

Por ejemplo, varias correcciones pequeñas pueden agruparse dentro de una versión de mantenimiento. De igual manera, varias mejoras de interfaz pueden formar parte de una versión menor.

Cuando los cambios sean suficientemente grandes, podrá considerarse una nueva versión mayor.

5. Registro y documentación de versiones

Cada nueva versión de EcoMuni GT deberá documentarse mediante un registro breve de cambios.

Este registro deberá indicar como mínimo el número de versión, fecha, tipo de actualización, mejoras realizadas, errores corregidos y nuevas funcionalidades incorporadas.

Un historial básico podría organizarse de la siguiente forma:

Versión 1.0.0: primera versión funcional del sistema.

Versión 1.1.0: correcciones y ajustes derivados de las pruebas.

Versión 1.2.0: mejoras de interfaz, navegación y experiencia del usuario.

Versión 2.0.0: ampliación futura con nuevos módulos y funcionalidades.

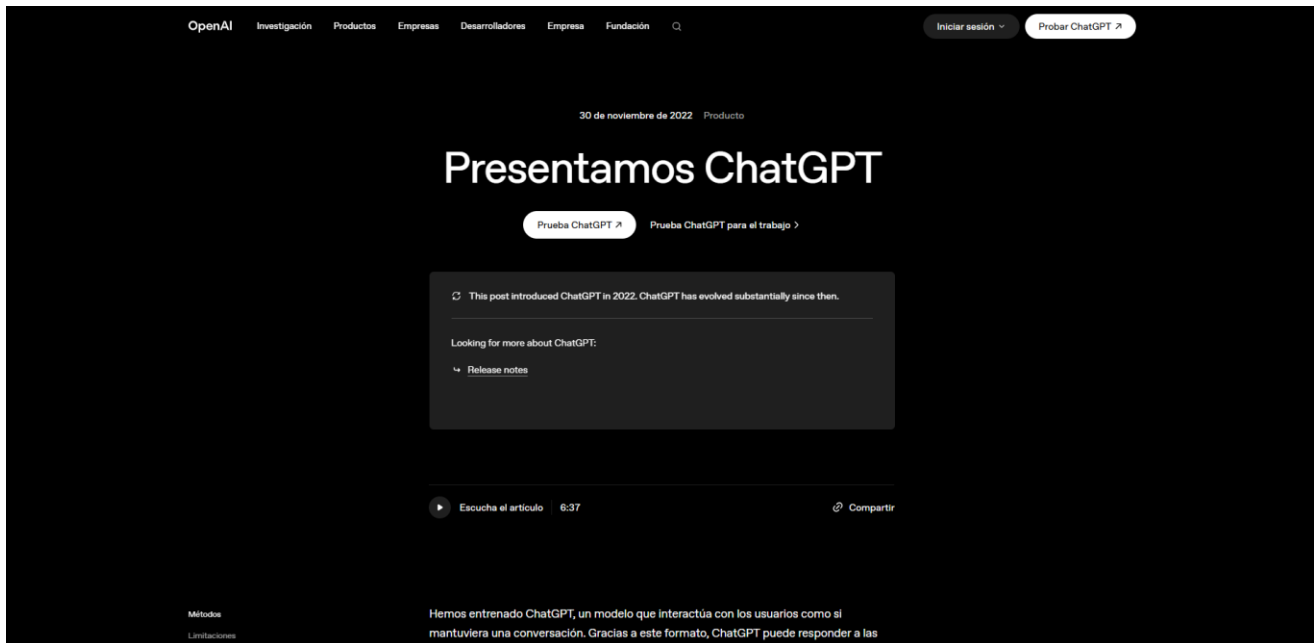
El registro de versiones permitirá conocer de manera clara cómo ha evolucionado el sistema y facilitará el trabajo de mantenimiento en etapas posteriores.

En conclusión, el versionado permite organizar el crecimiento de EcoMuni GT y mantener un historial comprensible de sus cambios. Gracias a este proceso, cada actualización podrá identificarse, compararse y documentarse de forma ordenada, asegurando una evolución controlada del sistema.

Uso responsable de la Inteligencia Artificial

Herramientas utilizadas

ChatGPT:



OpenCode:



Ejemplo de prompt usado:

Crea una barra de navegación moderna y elegante con efecto de desenfoque. Quiero que esté fija en la parte superior, centrada y con bordes redondeados. Usa colores suaves como beige claro, blanco y verde oscuro.

Debe incluir:

- Un logo con ícono y texto.
- Enlaces de navegación alineados horizontalmente.

- Efectos suaves al pasar el cursor.
- Un botón de cerrar sesión en color rojo.
- Animaciones y transiciones fluidas.
- Diseño limpio, profesional y moderno.

Genera el HTML y CSS completos utilizando buenas prácticas y un diseño responsive.

Resultado ya más arreglado del prompt anterior:

```
.navbar {
  position: fixed;
  top: 20px;
  left: 50%;
  transform: translateX(-50%);
  width: 92%;
  max-width: 1200px;
  height: 65px;
  background: rgba(215, 210, 199, 0.75);
  backdrop-filter: blur(10px);
  -webkit-backdrop-filter: blur(10px);
  border-radius: 40px;
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 0 25px;
  gap: 0;
  box-shadow: 0 8px 32px 0 rgba(0, 0, 0, 0.15);
  border: 1px solid rgba(255, 255, 255, 0.4);
  z-index: 10;
}

.logo-container {
  display: flex;
  align-items: center;
  gap: 10px;
  text-decoration: none;
  transition:
    transform 0.5s ease,
    opacity 0.5s ease;
}

.logo-container:hover {
  transform: translateY(-2px);
  opacity: 0.85;
}

.logo-icon {
  width: 32px;
  height: 32px;
  object-fit: contain;
}

.logo-text {
  font-size: 1.4rem;
  font-weight: bold;
  font-style: italic;
  color: #1f3b2c;
}
```

Modificaciones hechas luego del resultado del prompt:

```

}

.nav-item {
  text-decoration: none;
  color: #1f3b2c;
  font-weight: 600;
  font-size: 0.95rem;
  padding: 6px 14px;
  border-radius: 16px;
  border: 1px solid rgba(255, 255, 255, 0);
  background-color: transparent;
  cursor: pointer;
  transition:
    color 0.5s ease,
    background-color 0.5s ease,
    border-color 0.5s ease,
    box-shadow 0.5s ease,
    transform 0.5s ease;
}

.nav-item:hover {
  color: #4a3a2a;
  background-color: rgba(255, 255, 255, 0.3);
  border-color: rgba(255, 255, 255, 0.8);
  box-shadow: 0 2px 8px rgba(0, 0, 0, 0.05);
  transform: translateY(-2px);
}

.nav-item.active {
  background-color: #ffffff;
  color: #1f3b2c;
  border-color: #ffffff;
  box-shadow: 0 4px 6px rgba(0, 0, 0, 0.08);
}

.logout-btn {
  background-color: #8b2020;
  color: #ffffff;
  border: none;
  padding: 9px 22px;
  border-radius: 20px;
  font-weight: 600;
  cursor: pointer;
  box-shadow: 0 4px 10px rgba(139, 32, 32, 0.4);
  transition:
    background-color 0.5s ease,
    transform 0.5s ease,
    box-shadow 0.5s ease;
}

```

Se cambiaron colores de botones de cerrar sesión y se mejoró el transform del hover para los botones, también se agregó un color semi transparente para los botones de la navbar.

